Android Programming Practicals

Combined Text-Only Practical Manual

Combined from the supplied Android Programming Practical and Module 2 PDFs. Written content, filenames, instructions and code are included. Screenshots, photos and other embedded images have intentionally been omitted.

INDEX — CLICK A TOPIC TO JUMP TO IT

Module I: Android Fundamentals & UI Design

1. Develop a simple "Hello World" Android application demonstrating project creation, Android Studio interface, Android components, AVD creation, and application execution.

2. Develop an Android application demonstrating the use of Programming Resources (colors.xml, themes.xml, strings.xml, dimens.xml, drawable resources) and apply them in UI components.

3. Develop an Android application demonstrating complete Activity Lifecycle methods and analyze execution using Logcat.

4. Develop an Android application to create and integrate a Fragment and demonstrate Fragment lifecycle methods.

5. Develop an Android application implementing Fragment-based UI navigation between multiple screens.

6. Develop an Android application demonstrating communication between Activity and Fragment.

7. Develop an Android application using LinearLayout (Horizontal and Vertical) to design structured UI.

8. Develop an Android application using RelativeLayout to align UI components dynamically.

9. Develop an Android application using TableLayout to design a structured Login/Registration Form.

10. Develop an Android application using ScrollView to display long scrollable content.

11. Develop an Android application using AbsoluteLayout demonstrating fixed positioning.

12. Develop an Android application using FrameLayout to overlay UI elements.

13. Develop an Android application using ListView with ArrayAdapter and implement item click handling.

14. Develop an Android application using GridView with Custom Adapter.

15. Develop an Android application implementing RecyclerView as a modern alternative to ListView.

Module 2: Android System Components & Data Handling

Menus, Dialogs and UI Interaction

1. Develop an Android application implementing AppBar (Toolbar) and Options Menu.

2. Develop an Android application demonstrating AlertDialog and custom DialogFragment.

3. Develop an Android application implementing contextual menus and popup menus.

Intents, Events and Listeners

4. Develop an Android application implementing Explicit and Implicit Intents.

5. Develop an Android application demonstrating Event Handling using Listeners (OnClick, OnLongClick, etc.).

6. Develop an Android application using Adapters to populate Spinner or ListView dynamically.

Services, Notifications and Broadcast Receivers

7. Develop an Android application implementing Started and Bound Services.

8. Develop an Android application implementing Notifications using Notification Manager.

9. Develop an Android application implementing Static and Dynamic Broadcast Receivers.

Database and Networking

10. Develop an Android application implementing SQLite database CRUD operations.

11. Develop an Android application integrating SQLite with ListView or RecyclerView.

12. Develop an Android application to fetch and parse JSON data from a web service.

Concurrency, Media and Security

13. Develop an Android application demonstrating Threads and Handlers for background processing.

14. Develop an Android application implementing Media APIs and Telephone APIs.

15. Develop an Android application implementing Runtime Permissions and security handling in AndroidManifest.xml.

ANDROID PROGRAMMING PRACTICAL

Text extracted from the original PDF; embedded images are omitted.

Module I: Android Fundamentals & UI Design

Android Basics and Resources

1. Develop a simple "Hello World" Android application demonstrating project creation, Android Studio interface, Android components, AVD creation, and application execution.

2. Develop an Android application demonstrating the use of Programming Resources (colors.xml, themes.xml, strings.xml, dimens.xml, drawable resources) and apply them in UI components.

3. Develop an Android application demonstrating complete Activity Lifecycle methods and analyze execution using Logcat.

Fragments and Activity Interaction

4. Develop an Android application to create and integrate a Fragment and demonstrate Fragment lifecycle methods.

5. Develop an Android application implementing Fragment-based UI navigation between multiple screens.

6. Develop an Android application demonstrating communication between Activity and Fragment.

Basic Layout Management

7. Develop an Android application using LinearLayout (Horizontal and Vertical) to design structured UI.

8. Develop an Android application using RelativeLayout to align UI components dynamically.

9. Develop an Android application using TableLayout to design a structured Login/Registration Form.

Advanced Layout and View Handling

10. Develop an Android application using ScrollView to display long scrollable content.

11. Develop an Android application using AbsoluteLayout demonstrating fixed positioning.

12. Develop an Android application using FrameLayout to overlay UI elements.

Adapter Based Views

13. Develop an Android application using ListView with ArrayAdapter and implement item click handling.

14. Develop an Android application using GridView with Custom Adapter

15. Develop an Android application implementing RecyclerView as a modern alternative to ListView.

Module 2: Android System Components & Data Handling

Menus, Dialogs and UI Interaction

1. Develop an Android application implementing AppBar (Toolbar) and Options Menu.

2. Develop an Android application demonstrating AlertDialog and custom DialogFragment.

3. Develop an Android application implementing contextual menus and popup menus.

Intents, Events and Listeners

4. Develop an Android application implementing Explicit and Implicit Intents.

5. Develop an Android application demonstrating Event Handling using Listeners (OnClick, OnLongClick, etc.).

6. Develop an Android application using Adapters to populate Spinner or ListView dynamically.

Services, Notifications and Broadcast Receivers

7. Develop an Android application implementing Started and Bound Services.

8. Develop an Android application implementing Notifications using Notification Manager.

9. Develop an Android application implementing Static and Dynamic Broadcast Receivers.

Database and Networking

10. Develop an Android application implementing SQLite database CRUD operations.

11. Develop an Android application integrating SQLite with ListView or RecyclerView.

12. Develop an Android application to fetch and parse JSON data from a web

Concurrency, Media and Security

13. Develop an Android application demonstrating Threads and Handlers for background processing.

14. Develop an Android application implementing Media APIs and Telephone APIs.

15. Develop an Android application implementing Runtime Permissions and security handling in AndroidManifest.xml.

Practical 01

Develop a simple "Hello World" Android application demonstrating project creation, Android Studio interface, Android components, AVD creation, and application execution.

Project 1: Hello World

Project Name: HelloWorld

Template: Empty Views Activity

Project Architecture:

Total 5 files

I

Filename: MainActivity.java

```
package com.example.helloworld;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
}
}
```

Path: app>java>com.example.helloworld>MainActivity.java

II

Filename: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:gravity="center"
android:padding="16dp">
<TextView
android:id="@+id/text_hello"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Hello, World!"
android:textSize="24sp"
android:textStyle="bold"
android:textColor="#000000" />
</LinearLayout>
```

Path: app>res>layout>activity_main.xml

III

Filename: AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
```

package="com.example.helloworld">

```
<application
android:allowBackup="true"
android:icon="@mipmap/ic_launcher"
android:label="@string/app_name"
android:roundIcon="@mipmap/ic_launcher_round"
android:supportsRtl="true"
android:theme="@style/Theme.Helloworld">
<activity android:name=".MainActivity"
android:exported="true"> <!-- Added android:exported attribute -->
<intent-filter>
<action android:name="android.intent.action.MAIN" />
<category android:name="android.intent.category.LAUNCHER" />
</intent-filter>


</activity>
</application>
</manifest>
```

Path: app>manifest>AndroidManifest.xml

IV

Filename: colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<color name="black">#FF000000</color>
<color name="white">#FFFFFFFF</color>
<color name="purple_500">#6200EE</color>
<color name="purple_700">#3700B3</color>
</resources>
```

Path: app>res>values>colors.xml

V

Filename: styles.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<style name="Theme.Helloworld"
```

parent="Theme.MaterialComponents.DayNight.DarkActionBar">

<!-- Customize your theme here -->

```
<item name="colorPrimary">@color/purple_500</item>
<item name="colorPrimaryVariant">@color/purple_700</item>
<item name="colorOnPrimary">@color/white</item>
</style>
</resources>
```

Path: app>res>values>styles.xml

VI (Prebuild Project Code: To ensure proper project detail: No need to update until needed)

Filename: build.gradle.kts (Module:app)

```
plugins {
```

alias(libs.plugins.android.application)

```
}
android {
namespace = "com.example.helloworld"
compileSdk = 35
defaultConfig {
applicationId = "com.example.helloworld"
minSdk = 24
targetSdk = 35
versionCode = 1
versionName = "1.0"
```

testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"

```
}
buildTypes {
release {
```

isMinifyEnabled = false

proguardFiles(

getDefaultProguardFile("proguard-android-optimize.txt"),

"proguard-rules.pro"

)

```
}


}
compileOptions {
```

sourceCompatibility = JavaVersion.VERSION_11

targetCompatibility = JavaVersion.VERSION_11

```
}
}
dependencies {
implementation(libs.appcompat)
implementation(libs.material)
implementation(libs.activity)
implementation(libs.constraintlayout)
testImplementation(libs.junit)
androidTestImplementation(libs.ext.junit)
androidTestImplementation(libs.espresso.core)
}
```

Path: Gradle Scripts > build.gradle.kts

Above the Android Studio near Main Menu Click the Play button to Run the Program

Recommended compulsory AVD Emulator: Pixel 4 API 25 with version 7.1.1 Nougat

Output:

b.Develop an Android application demonstrating the use of Programming Resources

(colors.xml, themes.xml, strings.xml, dimens.xml, drawable resources) and apply them in

UI components.

Programming Resources

Android Resources: (Color, Theme, String, Drawable, Dimension, Image)

Project Name: Practical 2R

Template: Empty Views Activity

Language: Java

Project Architecture: 1 AndroidManifest.xml file, 6 Java Files,1 image in drawable image

file 6 layout .xml files and 4 values .xml files: Total 18 files.

Filename: AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools">
<application
android:allowBackup="true"
android:dataExtractionRules="@xml/data_extraction_rules"
android:fullBackupContent="@xml/backup_rules"
android:icon="@mipmap/ic_launcher"
android:label="@string/app_name"
android:roundIcon="@mipmap/ic_launcher_round"
android:supportsRtl="true"
android:theme="@style/Theme.AppCompat.Light"
```

tools:targetApi="31">

```
<activity android:name=".ThemeActivity" android:label="Theme Demo" />
<activity android:name=".StringActivity" android:label="String Demo" />
<activity android:name=".DrawableActivity" android:label="Drawable Demo"
/>
<activity android:name=".DimensionActivity" android:label="Dimension
Demo" />
<activity android:name=".ColorActivity" android:label="Color Demo" />
<activity
android:name=".MainActivity"
android:exported="true"
android:label="@string/app_name"
android:theme="@style/Theme.AppCompat.Light">
<intent-filter>
<action android:name="android.intent.action.MAIN" />


<category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
</activity>
</application>
</manifest>
```

Path: app>manifests>AndroidManifest.xml

Total 6 .java files path

1.Filename: ColorActivity.java

```
package com.example.practical2r;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class ColorActivity extends AppCompatActivity {
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_color);
}
}
```

2.Filename: DimensionActivity.java

```
package com.example.practical2r;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class DimensionActivity extends AppCompatActivity {
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_dimension);
}
}
```

3.Filename: DrawableActivity.java

```
package com.example.practical2r;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
```

```
public class DrawableActivity extends AppCompatActivity {
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_drawable);
}
}
```

4.Filename: MainActivity.java

```
package com.example.practical2r;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
}
public void openColorActivity(View view) {
startActivity(new Intent(this, ColorActivity.class));
}
public void openThemeActivity(View view) {
startActivity(new Intent(this, ThemeActivity.class));
}
public void openStringActivity(View view) {
startActivity(new Intent(this, StringActivity.class));
}
public void openDrawableActivity(View view) {
startActivity(new Intent(this, DrawableActivity.class));
}
public void openDimensionActivity(View view) {
startActivity(new Intent(this, DimensionActivity.class));
}
}
```

5.Filename: StringActivity.java

```
package com.example.practical2r;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class StringActivity extends AppCompatActivity {
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_string);
}
}
```

6.Filename: ThemeActivity.java

```
package com.example.practical2r;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class ThemeActivity extends AppCompatActivity {
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_theme);
}
}
```

Image File Path

Image Detail: pebbles.jpg

(Copy your image file from your local folder and only copy and paste into drawable folder.

Don’t Drag and Drop). Image size must within KB size

Total 6 Layout .xml files path

1.Filename: activity_color.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:background="@color/colorRed">
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Color Activity"
```

```
android:textSize="24sp"
android:layout_centerInParent="true"
android:textColor="@color/white"/>
</RelativeLayout>
```

2.Filename: activity_dimension.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="16dp"
android:gravity="center">
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Dimension Activity"
android:textSize="24sp"


android:textColor="@android:color/black"/>
<!-- Button demonstrating dimension usage -->
<Button
android:id="@+id/dimensionButton"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Click Me!"
android:textSize="@dimen/button_text_size"
android:padding="@dimen/button_padding"
android:layout_marginTop="@dimen/button_margin_top"
android:background="@color/btn_color"/>
</LinearLayout>
```

3.Filename: activity_drawable.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:background="@drawable/pebbles">
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Drawable Activity"
android:textSize="24sp"
android:layout_centerInParent="true"
android:textColor="@android:color/white"/>
</RelativeLayout>
```

4.Filename: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
```

tools:context=".MainActivity"

```
android:background="@drawable/pebbles">
<Button
android:id="@+id/btnColor"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Color Demo"
android:layout_centerHorizontal="true"
android:layout_marginTop="50dp"
android:onClick="openColorActivity"/>
<Button
android:id="@+id/btnTheme"


android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Theme Demo"
android:layout_below="@id/btnColor"
android:layout_centerHorizontal="true"
android:layout_marginTop="20dp"
android:onClick="openThemeActivity"/>
<Button
android:id="@+id/btnString"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="String Demo"
android:layout_below="@id/btnTheme"
android:layout_centerHorizontal="true"
android:layout_marginTop="20dp"
```

```
android:onClick="openStringActivity"/>
<Button
android:id="@+id/btnDrawable"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Drawable Demo"
android:layout_below="@id/btnString"
android:layout_centerHorizontal="true"
android:layout_marginTop="20dp"
android:onClick="openDrawableActivity"/>
<Button
android:id="@+id/btnDimension"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Dimension Demo"
android:layout_below="@id/btnDrawable"
android:layout_centerHorizontal="true"
android:layout_marginTop="20dp"
android:onClick="openDimensionActivity"/>
</RelativeLayout>
```

5.Filename: activity_string.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:gravity="center"
android:background="@color/colorPrimaryDark">
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="@string/Welcome_Message"
android:textSize="24sp"
android:textColor="@android:color/white"/>
</LinearLayout>
```

6.Filename: activity_theme.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:background="@android:color/holo_blue_bright">
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="You got this sky blue theme"
android:textSize="24sp"
android:layout_centerInParent="true"
android:textColor="@android:color/black"/>
</RelativeLayout>
```

Total 4 values .xml files

1.Filename: colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<color name="purple_200">#FFBB86FC</color>
<color name="purple_500">#FF6200EE</color>
<color name="purple_700">#FF3700B3</color>
<color name="teal_200">#9ADA03</color>
<color name="teal_700">#FF018786</color>
<color name="black">#FF000000</color>
<color name="white">#FFFFFFFF</color>
<color name="btn_color">#FFC49662</color>
<color name="colorRed">#E91E63</color>
<color name="colorBlue">#0000FF</color>
<color name="colorPrimary">#6200EE</color>
<color name="colorPrimaryDark">#3700B3</color>
<color name="colorAccent">#03DAC5</color>
</resources>
```

2.Filename: dimens.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<dimen name="button_margin_top">24dp</dimen>
<dimen name="button_padding">16dp</dimen>
<dimen name="button_text_size">18sp</dimen>
</resources>
```

3.Filename: strings.xml

```
<resources>
<string name="app_name">Practical 2R</string>
<string name="button_name">Click on me</string>
<string name="ParaHead">Welcome to Practical 2R</string>
<string name="Description">This is a string resource demo.</string>
<string name="Welcome_Message">Welcome to Android Studio. This is your
```

Practical 2 Resources.</string>

```
</resources>
```

4.Filename: themes.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<style name="AppTheme" parent="Theme.AppCompat.Light.DarkActionBar">
<item name="colorPrimary">@color/colorPrimary</item>
<item name="colorPrimaryDark">@color/colorPrimaryDark</item>
<item name="colorAccent">@color/colorAccent</item>
</style>
<style name="BlueTheme" parent="Theme.AppCompat.Light.DarkActionBar">
<item name="colorPrimary">@color/colorPrimary</item>
<item name="colorPrimaryDark">@color/colorPrimaryDark</item>
<item name="colorAccent">@color/colorAccent</item>
<item name="android:background">#00F0EF</item>
<item name="android:textColor">#000000</item>
</style>
</resources>
```

Above the Android Studio near Main Menu Click the Play button to Run the Program

Recommended compulsory AVD Emulator: Pixel 4 API 25 with version 7.1.1 Nougat

Play Emulator

Output:

MAIN PAGE

Click COLOR DEMO press back & Click THEME DEMO press back & Click THEME DEMO

Press back &Click COLOR DEMO Press back & Click COLOR DEMO

C. Develop an Android application demonstrating complete Activity Lifecycle methods

and analyze execution using Logcat.

Project Name: logcat

```
package com.example.logact;
import android.os.Bundle;
import android.util.Log;
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
public class MainActivity extends AppCompatActivity {
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
}
@Override
protected void onStart() {
super.onStart();
Log.d("on start invoked", "on started");
}
@Override
protected void onPostResume() {
super.onPostResume();
Log.d("on resume invoked", "on resume");
}
@Override
protected void onPause() {
super.onPause();
Log.d("on pause invoked", "on pause");
}
@Override
protected void onStop() {
```

super.onStop();

Log.d("on stop invoked", "on stopped");

```
}
@Override
protected void onDestroy() {
super.onDestroy();
Log.d("on destroy invoked", "on destroy");
}
}
```

2. Fragments and Activity Interaction

4. Develop an Android application to create and integrate a Fragment and demonstrate

Fragment lifecycle methods.

Project Name: FragmentLifecycleDemo

Create a new fragment class:

java->new-> fragment->fragment(blank)

Fragment Name: DemoFragment

Fragment Layout name: fragment_demo

I

Filename: DemoFragment.java

```
package com.example.fragmentlifecycledemo;
import android.content.Context;
import android.os.Bundle;
import android.util.Log;
import android.view.LayoutInflater; import android.view.View;
import android.view.ViewGroup;
import androidx.annotation.NonNull; import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
public class DemoFragment extends Fragment {
String TAG = "FragmentLifecycle";
@Override
public void onAttach(@NonNull Context context) {
super.onAttach(context);
```

Log.d(TAG, "onAttach() executed");}

```
@Override
public void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
Log.d(TAG, "onCreate() executed");
}
@Override
public View onCreateView(LayoutInflater inflater, ViewGroup
container,Bundle savedInstanceState) {
Log.d(TAG, "onCreateView() executed");
return inflater.inflate(R.layout.fragment_demo, container, false);
}
@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle
savedInstanceState) {
super.onViewCreated(view, savedInstanceState);
Log.d("on view created invoked", "onViewCreated() executed");
}
@Override
public void onStart() {
super.onStart();
Log.d("on start invoked", "on started");


}
@Override
public void onResume() {
super.onResume();
Log.d("on resume invoked", "on resume");
}
@Override
public void onPause() {
super.onPause();
Log.d("on pause invoked", "on pause");
}
@Override
public void onStop() {
```

```
super.onStop();
Log.d("on stop invoked", "on stopped");
}
@Override
public void onDestroyView() {
super.onDestroyView();
Log.d("on destroyview invoked", "on destroyview() executed");
}


@Override
public void onDestroy() {
super.onDestroy();
Log.d("on destroy invoked", "on destroy");
}
@Override
public void onDetach() {
super.onDetach();
Log.d("on detach invoked", "on detach");
}
}
```

Filename: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/fragment_container_view_tag"
android:layout_width="match_parent"
android:layout_height="match_parent"
```

tools:context=".MainActivity">

```
<TextView


android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Hello World!"
app:layout_constraintBottom_toBottomOf="parent"
app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Filename: MainActivity.java

```
package com.example.fragmentlifecycledemo;
import android.os.Bundle;
import androidx.fragment.app.FragmentTransaction;
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
public class MainActivity extends AppCompatActivity {
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
```

setContentView(R.layout.activity_main);

DemoFragment fragment = new DemoFragment();

FragmentTransaction transaction =

getSupportFragmentManager().beginTransaction();

transaction.replace(R.id.fragment_container_view_tag,

fragment);

transaction.commit();

```
}
}
```

fragment_demo.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
```

tools:context=".DemoFragment">

<!-- TODO: Update blank fragment layout -->

```
<TextView
android:layout_width="match_parent"
android:layout_height="match_parent"
android:text="@string/hello_blank_fragment" />
</FrameLayout>
```

5. Develop an Android application implementing Fragment-based UI navigation between

multiple screens.

Activity Life Cycle, Activity methods, Multiple Activities, Life Cycle of fragments and

multiple fragments.

Project Name: Fragment Lifecycle

Template: Empty Views Activity

Language: Java

Project Architecture: Total 6 files

I Filename: MainActivity.java

```
package com.example.fragmentlifecycle;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.widget.Button;
public class MainActivity extends AppCompatActivity {
private boolean isFragmentOneLoaded = true;
private final androidx.fragment.app.FragmentManager fragmentManager =
getSupportFragmentManager();
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
```

Button btnChange = findViewById(R.id.btn_change); // Ensure ID

matches activity_main.xml

loadFragmentOne();

btnChange.setOnClickListener(v -> {

if (isFragmentOneLoaded) {

loadFragmentTwo();

} else {

loadFragmentOne();

```
}
});
}
private void loadFragmentOne() {
androidx.fragment.app.FragmentTransaction transaction =
fragmentManager.beginTransaction();
FragmentOne fragment = new FragmentOne();
```

transaction.replace(R.id.fragment_holder, fragment); // Ensure ID

matches activity_main.xml

transaction.addToBackStack(null);

transaction.commit();

isFragmentOneLoaded = true;

```
}
private void loadFragmentTwo() {
androidx.fragment.app.FragmentTransaction transaction =
fragmentManager.beginTransaction();
FragmentTwo fragment = new FragmentTwo();
```

transaction.replace(R.id.fragment_holder, fragment); // Ensure ID

matches activity_main.xml

transaction.addToBackStack(null);

transaction.commit();

isFragmentOneLoaded = false;

```
}
}
```

Path: app>java>com.example.fragmentlifecycle>MainActivity.java

II

Filename: FragmentOne.java

```
package com.example.fragmentlifecycle;
import android.content.Context;
import android.os.Bundle;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import androidx.fragment.app.Fragment;
public class FragmentOne extends Fragment {
private static final String TAG = "FragmentOne";
@Override
public void onAttach(Context context) {
super.onAttach(context);
```

Log.d(TAG, "onAttach");

```
}
@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container, Bundle
savedInstanceState) {
Log.d(TAG, "onCreateView");
return inflater.inflate(R.layout.fragment_one, container, false);
}
}
```

Path: app>java>com.example.fragmentlifecycle > FragmentOne.java

III

Filename: FragmentTwo.java

```
package com.example.fragmentlifecycle;
import android.content.Context;
import android.os.Bundle;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import androidx.fragment.app.Fragment;
public class FragmentTwo extends Fragment {
private static final String TAG = "FragmentTwo";


@Override
public void onAttach(Context context) {
super.onAttach(context);
Log.d(TAG, "onAttach");
}\
@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container, Bundle
savedInstanceState) {
Log.d(TAG, "onCreateView");
return inflater.inflate(R.layout.fragment_two, container, false);
}
}
```

Path: app>java>com.example.fragmentlifecycle > FragmentTwo.java

IV

Filename: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
```

tools:context=".MainActivity">

<!-- Button with ID btn_change -->

<!-- FrameLayout with ID fragment_holder -->

<FrameLayout

```
android:id="@+id/fragment_holder"
android:layout_width="match_parent"
android:layout_height="0dp"
android:layout_weight="2">
<Button
android:id="@+id/btn_change"
android:layout_width="365dp"
android:layout_height="88dp"
android:layout_margin="15dp"
android:layout_marginTop="15dp"
android:backgroundTint="@android:color/holo_green_dark"
android:text="Change Fragment"
android:textAllCaps="false"
android:textSize="24sp" />
</FrameLayout>
</LinearLayout>
```

Path: app>res>layout>activity_main.xml

V

Filename: fragment_one.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent">
<TextView
android:id="@+id/textView"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_centerInParent="true"
android:text="Fragment One"
android:textSize="24sp" />
</RelativeLayout>
```

Path: app>res>layout>fragment_one.xml

VI

Filename: fragment_two.xml

```
<?xml version="1.0" encoding="utf-8"?>


<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent">
<TextView
android:id="@+id/textView2"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_centerInParent="true"
android:text="Fragment Two"
android:textSize="24sp" />
</RelativeLayout>
```

Path: app>res>layout>fragment_two.xml

Above the Android Studio near Main Menu Click the Play button to Run the Program

6. Develop an Android application demonstrating communication between Activity and

Fragment.

Project Name: ActivityFragmentCommunication

Creating Fragment

java package → New→ Fragment → Fragment (Blank)

Fragment Name: DemoFragment

Layout Name: fragment_demo

Filename: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:id="@+id/fragment_container"
android:layout_width="match_parent"
android:layout_height="match_parent"/>
```

Open From res/layout:

Filename: fragment_demo.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:gravity="center"
android:orientation="vertical"
android:padding="20dp">
<TextView


android:id="@+id/txtMessage"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:textSize="22sp"
android:text="Message from Activity"/>
<Button
android:id="@+id/btnSend"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Send Message to Activity"
android:layout_marginTop="20dp"/>
</LinearLayout>
```

Modify DemoFragment.java.

Filename: DemoFragment.java

```
package com.example.activityfragmentcommunication;
import android.content.Context;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup; import android.widget.Button;
import android.widget.TextView;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;


import androidx.fragment.app.Fragment;
public class DemoFragment extends Fragment
{
TextView txtMessage;
Button btnSend;
String activityMessage;
FragmentCommunication communication;
public interface FragmentCommunication {
void onMessageFromFragment(String message);
}
// Receive Activity object
@Override
public void onAttach(@NonNull Context context) {
super.onAttach(context);
communication = (FragmentCommunication) context;
}
// Method to receive data from Activity
public static DemoFragment newInstance(String message) {
DemoFragment fragment = new DemoFragment();
Bundle bundle = new Bundle();
bundle.putString("msg", message);
```

fragment.setArguments(bundle);

return fragment;

```
}
@Override
public View onCreateView(LayoutInflater inflater,ViewGroup container,Bundle
savedInstanceState) {
View view = inflater.inflate(R.layout.fragment_demo,container, false);
txtMessage = view.findViewById(R.id.txtMessage);
btnSend = view.findViewById(R.id.btnSend);
// Receiving data from Activity
if(getArguments() != null){
activityMessage = getArguments().getString("msg");
txtMessage.setText(activityMessage);
}
// Sending data to Activity
btnSend.setOnClickListener(v -> {
```

communication.onMessageFromFragment(

"Hello Activity! Message from Fragment");

```
}
```

```
);
return view;
}

}
```

Filename: MainActivity.java

```
package com.example.activityfragmentcommunication;
import android.os.Bundle;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity
implements DemoFragment.FragmentCommunication {
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
// Sending data from Activity to Fragment
```

DemoFragment fragment = DemoFragment.newInstance(

"Hello Fragment! Message from Activity"

);

getSupportFragmentManager()

.beginTransaction()

.replace(R.id.fragment_container, fragment)

.commit();

```
}
// Receiving data from Fragment
@Override
public void onMessageFromFragment(String message) {
Toast.makeText(this, message, Toast.LENGTH_LONG).show();
}
}
```

OUTPUT:

III. Basic Layout Management

7. Develop an Android application using LinearLayout (Horizontal and Vertical) to design

structured UI.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:id="@+id/main"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="20dp">
<!-- Title -->
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Student Registration" android:textSize="26sp"
android:textStyle="bold"
android:layout_gravity="center" android:layout_marginBottom="20dp"/>
<!--Vertical Layout Components -->
<EditText
android:id="@+id/etName"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Enter Name"
android:layout_marginBottom="10dp"/>
```

<EditText

```
android:id="@+id/etEmail"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Enter Email"
android:layout_marginBottom="20dp"/>
<!-- Horizontal LinearLayout -->
<LinearLayout
```

```
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="horizontal"
android:gravity="center">
<Button
android:id="@+id/btnSubmit"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Submit"/>
<Button
android:id="@+id/btnReset"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Reset"
android:layout_marginStart="20dp"/>
</LinearLayout>


</LinearLayout>
```

MODULE 2 PRACTICALS

Text extracted from the original PDF; embedded images are omitted.

Menus, Dialogs and UI Interaction

1. Develop an Android application implementing AppBar (Toolbar) and Options Menu.

Project Name: ToolbarMenuDemo

FileName: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
android:id="@+id/main"
android:layout_width="match_parent"
android:layout_height="match_parent" android:orientation="vertical">
<!-- Toolbar -->
<androidx.appcompat.widget.Toolbar
android:id="@+id/toolbar"
android:layout_width="match_parent"
android:layout_height="?attr/actionBarSize"
android:background="?attr/colorPrimary"
app:title="Toolbar Demo"
app:titleTextColor="@android:color/white"/>
<!--Main Content -->
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="AppBar and Options Menu Example"


android:textSize="22sp"
android:padding="20dp"/>
</LinearLayout>
```

Create a Menu Resource File.

Right Click: Res → New→ Android Resource Directory

Directory Name: menu

Resource Type: menu

Create Menu XML File.

Right Click: menu New→ Menu Resource File Give name as menu_main.xml

File Name : menu_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto">
<item
android:id="@+id/menu_home"
android:title="Home"
app:showAsAction="never"/>
<item
android:id="@+id/menu_settings"
android:title="Settings"
app:showAsAction="never"/>
<item
android:id="@+id/menu_about"
android:title="About"
app:showAsAction="never"/>
<item
android:id="@+id/menu_logout"
android:title="Logout"
app:showAsAction="never"/>
</menu>
```

File Name: MainActivity.java.

```
package com.example.toolbarmenudemo;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.widget.Toolbar;
public class MainActivity extends AppCompatActivity {
Toolbar toolbar;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
toolbar = findViewById(R.id.toolbar);
// Set Toolbar as AppBar
setSupportActionBar(toolbar);
}
// Create Options Menu
@Override
public boolean onCreateOptionsMenu(Menu menu) {
```

getMenuInflater().inflate(R.menu.menu_main, menu);

return true;

```
}
// Handle Menu Item Click
@Override
public boolean onOptionsItemSelected(@NonNull MenuItem item) {
int id = item.getItemId();
if (id == R.id.menu_home) {
Toast.makeText(this, "Home Selected", Toast.LENGTH_SHORT).show();
} else if (id == R.id.menu_settings) {
Toast.makeText(this, "Settings Selected",
Toast.LENGTH_SHORT).show();
} else if (id == R.id.menu_about) {
Toast.makeText(this, "About Selected", Toast.LENGTH_SHORT).show();
} else if (id == R.id.menu_logout) {
Toast.makeText(this, "Logout Selected", Toast.LENGTH_SHORT).show();
}
return true;
}
}
```

2. Develop an Android application demonstrating AlertDialog and custom

DialogFragment.

ProjectName: DialogDemo

File Name: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:id="@+id/main"
android:layout_width="match_parent" android:layout_height="match_parent"
android:orientation="vertical"


android:gravity="center" android:padding="20dp">
<Button
android:id="@+id/btnAlert"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Show AlertDialog"/>
<Button
android:id="@+id/btnCustom" android:layout_width="wrap_content"
android:layout_height="wrap_content" android:text="Show Custom Dialog"
android:layout_marginTop="20dp"/>
</LinearLayout>
```

Create Custom Dialog Layout.

Create:

res→ layout → New-> XML-> New Layout XML File → dialog_custom

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
```

```
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="vertical"
android:padding="20dp">
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Custom DialogFragment"
android:textSize="22sp"
android:textStyle="bold"/>
<EditText
android:id="@+id/etName"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Enter Name"
android:layout_marginTop="15dp"/>
</LinearLayout>
```

Create Dialog Fragment Class.

Right Click: java package → New → Java Class

Class Name: CustomDialogFragment.java

```
package com.example.dialogdemo;
import android.app.AlertDialog;
import android.app.Dialog;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.widget.EditText;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.fragment.app.DialogFragment;
public class CustomDialogFragment extends DialogFragment {
EditText etName;
@NonNull
@Override
public Dialog onCreateDialog(Bundle savedInstanceState) {
AlertDialog.Builder builder = new AlertDialog.Builder(getActivity());
LayoutInflater inflater = requireActivity().getLayoutInflater();
View view = inflater.inflate(R.layout.dialog_custom, null);
```

etName = view.findViewById(R.id.etName);

builder.setView(view)

.setTitle("Custom Dialog")

.setPositiveButton("Submit", (dialog, which) -> {

String name = etName.getText().toString();

Toast.makeText(getActivity(),

"Entered: " + name,

Toast.LENGTH_SHORT).show();

})

.setNegativeButton("Cancel", (dialog, which) ->

dialog.dismiss());

return builder.create();

```
}
}
```

FileName: MainActivity.java

```
package com.example.dialogdemo;
import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;


public class MainActivity extends AppCompatActivity {
Button btnAlert, btnCustom;
@Override
protected void onCreate(Bundle savedInstanceState) {
```

```
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
btnAlert = findViewById(R.id.btnAlert);
btnCustom = findViewById(R.id.btnCustom);
// AlertDialog Example
btnAlert.setOnClickListener(v -> {
AlertDialog.Builder builder = new AlertDialog.Builder(this);
builder.setTitle("Delete File")
```

.setMessage("Are you sure you want to delete?")

.setPositiveButton("YES", (dialog, which) -> {

Toast.makeText(this,

"File Deleted",

Toast.LENGTH_SHORT).show();

})

.setNegativeButton("NO", (dialog, which) ->

dialog.dismiss());

builder.show();

```
});
// Custom Dialog Example
btnCustom.setOnClickListener(v -> {
CustomDialogFragment dialog = new CustomDialogFragment();
dialog.show(getSupportFragmentManager(), "CustomDialog");
});
}
}
```

3. Develop an Android application implementing contextual menus and popup menus.

ProjectName: MenuDemo

File Name: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:gravity="center"
android:padding="20dp">
<TextView
android:id="@+id/txtContext"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Long Press Me"
android:textSize="24sp"
android:padding="20dp"/>
<Button
android:id="@+id/btnPopup"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Show Popup Menu"
android:layout_marginTop="30dp"/>
</LinearLayout>
```

Create a Menu Resource File.

Right Click: res→ New → Android Resource Directory

Directory Name: menu

Resource Type: menu

Click OK

Right Click: menu → New → Menu Resource File

File Name : menu_options

File Name : menu_options.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
<item
android:id="@+id/menu_edit" android:title="Edit"/>
<item
android:id="@+id/menu_delete"
android:title="Delete"/>
<item
android:id="@+id/menu_share"
android:title="Share"/>
</menu>
```

File Name: MainActivity.java

```
package com.example.menudemo;
import android.os.Bundle;
import android.view.ContextMenu;
import android.view.MenuItem;
import android.view.View;
import android.widget.Button;
import android.widget.PopupMenu;
import android.widget.TextView;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;


public class MainActivity extends AppCompatActivity {
TextView txtContext;
Button btnPopup;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
txtContext = findViewById(R.id.txtContext);
btnPopup = findViewById(R.id.btnPopup);
// Register Context Menu
registerForContextMenu(txtContext);
// Popup Menu
btnPopup.setOnClickListener(v -> {
PopupMenu popupMenu = new PopupMenu(MainActivity.this, btnPopup);
popupMenu.getMenuInflater().inflate(
```

R.menu.menu_options,

popupMenu.getMenu());

popupMenu.setOnMenuItemClickListener(item -> {

Toast.makeText(MainActivity.this,

item.getTitle(),

Toast.LENGTH_SHORT).show();

return true;

```
});
popupMenu.show();
});
}
// Context Menu Creation
@Override
public void onCreateContextMenu(ContextMenu menu,
```

View v,

ContextMenu.ContextMenuInfo menuInfo) {

super.onCreateContextMenu(menu, v, menuInfo);

menu.setHeaderTitle("Select Option");

getMenuInflater().inflate(R.menu.menu_options, menu);

```
}
// Context Menu Click Handling
@Override
public boolean onContextItemSelected(@NonNull MenuItem item) {
```

Toast.makeText(this,

item.getTitle(),

Toast.LENGTH_SHORT).show();

return true;

```
}
}
```

Intents, Events and Listeners

4. Develop an Android application implementing Explicit and Implicit Intents.

ProjectName: IntentDemo

File Name: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent" android:layout_height="match_parent"
android:orientation="vertical"
android:gravity="center"
android:id="@+id/main"
android:padding="20dp">
<Button
android:id="@+id/btnExplicit"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Explicit Intent"/>
<Button
android:id="@+id/btnImplicit"
android:layout_width="wrap_content"
android:layout_height="wrap_content" android:text="Implicit Intent"
android:layout_marginTop="20dp"/>
</LinearLayout>
```

Create Second Activity.

Right Click: java package → com.example.projectname -> New→ Activity → Empty Views

Activity

Activity Name: SecondActivity

Click: Finish

Design Second Activity Layout.

Open: res→ layout → activity_second.xml

File Name: activity_second.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:id="@+id/main"
android:gravity="center">
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Welcome to Second Activity"
android:textSize="24sp"/>
</LinearLayout>
```

File Name: MainActivity.java.

```
package com.example.intentdemo;
import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
Button btnExplicit, btnImplicit;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
btnExplicit = findViewById(R.id.btnExplicit);
btnImplicit = findViewById(R.id.btnImplicit);
// Explicit Intent
btnExplicit.setOnClickListener(v -> {
Intent intent = new Intent(MainActivity.this, SecondActivity.class);
startActivity(intent);
});


// Implicit Intent
btnImplicit.setOnClickListener(v -> {
```

Intent intent = new Intent(

Intent.ACTION_VIEW,

Uri.parse("https://www.google.com")

);

startActivity(intent);

```
});
}
}
```

5. Develop an Android application demonstrating Event Handling using Listeners

(OnClick, OnLongClick, etc.).

Project Name: EventHandlingDemo

File Name: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:gravity="center"
android:padding="20dp">
<Button
android:id="@+id/btnClick"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Click Me"/>
<Button
android:id="@+id/btnLongClick"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Long Press Me"
android:layout_marginTop="20dp"/>
</LinearLayout>
```

File Name: MainActivity.java

```
package com.example.eventhandlingdemo;
import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
Button btnClick, btnLongClick;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
btnClick = findViewById(R.id.btnClick);
btnLongClick = findViewById(R.id.btnLongClick);
// OnClickListener
btnClick.setOnClickListener(v -> {
Toast.makeText(
```

MainActivity.this,

"Button Clicked",

Toast.LENGTH_SHORT

).show();

```
});
// OnLongClickListener
btnLongClick.setOnLongClickListener(v -> {
Toast.makeText(
```

MainActivity.this,

"Button Long Pressed",

Toast.LENGTH_SHORT

).show();

return true;

```
});
}
}
```

6. Develop an Android application using Adapters to populate Spinner or ListView dynamically.

Android Spinner is a view similar to the dropdown list which is used to select one option from the list of options. It provides an easy way to select one item from the list of items and it shows a dropdown list of all values when we click on it. The default value of the android spinner will be the currently selected value and by using Adapter we can easily bind the items to the spinner objects. Generally, we populate our Spinner control with a list of items by using an ArrayAdapter in our Kotlin/Java file.

Project Name: AdapterDemo

File Name: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="20dp">
<Spinner
android:id="@+id/spinner"
android:layout_width="match_parent"
android:layout_height="wrap_content"/>
<ListView
android:id="@+id/listView"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:layout_marginTop="20dp"/>
</LinearLayout>
```

File Name: MainActivity.java

```
package com.example.adapterdemo;
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.Spinner;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
Spinner spinner;
ListView listView;
String[] cities = {
```

"Mumbai",

"Pune",

"Delhi",

"Chennai",

"Hyderabad"

};

```
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
spinner = findViewById(R.id.spinner);
listView = findViewById(R.id.listView);
// Spinner Adapter
```

ArrayAdapter<String> spinnerAdapter = new ArrayAdapter<>(

this,

android.R.layout.simple_spinner_dropdown_item,

cities

);

spinner.setAdapter(spinnerAdapter);

```
// ListView Adapter
```

ArrayAdapter<String> listAdapter = new ArrayAdapter<>(

this,

android.R.layout.simple_list_item_1,

cities

);

listView.setAdapter(listAdapter);

```
// Spinner Item Selection
```

spinner.setOnItemSelectedListener(new

AdapterView.OnItemSelectedListener() {

```
@Override
public void onItemSelected(AdapterView<?> parent, View view,
int position, long id) {
Toast.makeText(
```

MainActivity.this,

"Selected: " + cities[position],

Toast.LENGTH_SHORT

).show();

```
}
@Override
public void onNothingSelected(AdapterView<?> parent) {
}
});
// ListView Item Click
```

```
listView.setOnItemClickListener((parent, view, position, id) -> {
Toast.makeText(
```

MainActivity.this,

"Clicked: " + cities[position],

Toast.LENGTH_SHORT

).show();

```
});
}
}
```

Services, Notifications and Broadcast Receivers

7. Develop an Android application implementing Started and Bound Services.

A bound service is the server in a client-server interface. It lets components such as activities bind to the service, send requests, receive responses, and perform interprocess communication (IPC). A bound service typically lives only while it serves another application component and does not run in the background indefinitely.

Project Name: ServiceDemo

File Name: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:gravity="center"
android:padding="20dp">
<Button
android:id="@+id/btnStart"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Start Service"/>
<Button
android:id="@+id/btnStop"
android:layout_width="wrap_content"
android:layout_height="wrap_content"


android:text="Stop Service"
android:layout_marginTop="15dp"/>
<Button
android:id="@+id/btnBound"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Bind Service"
android:layout_marginTop="15dp"/>
</LinearLayout>
```

Create a Started Service Class.

Right Click: java package→ New → Java Class

Class Name: MyStartedService

File Name: MyStartedService.java

```
package com.example.servicedemo;
import android.app.Service;
import android.content.Intent;
import android.os.IBinder;
import android.widget.Toast;
public class MyStartedService extends Service {
@Override
public int onStartCommand(Intent intent, int flags, int startId) {
```

Toast.makeText(

this,

"Started Service Running",

Toast.LENGTH_SHORT

).show();

return START_STICKY;

```
}
@Override
public void onDestroy() {
super.onDestroy();
Toast.makeText(
```

this,

"Started Service Stopped",

Toast.LENGTH_SHORT

).show();

```
}
@Override
public IBinder onBind(Intent intent) {
return null;
}
}
```

Create a Bound Service Class (Similar like above).

Create Java Class: MyBoundService

File Name:MyBoundService

```
package com.example.servicedemo;
import android.app.Service;
import android.content.Intent;
import android.os.Binder;
import android.os.IBinder;
public class MyBoundService extends Service {
private final IBinder binder = new LocalBinder();
public class LocalBinder extends Binder {
public MyBoundService getService() {
return MyBoundService.this;
}
}
@Override
public IBinder onBind(Intent intent) {
return binder;
}
public String getMessage() {
```

return "Bound Service Connected";

```
}
}
```

File Name: MainActivity.java

```
package com.example.servicedemo;
import android.content.ComponentName;
import android.content.Intent;
import android.content.ServiceConnection;
import android.os.Bundle;
import android.os.IBinder;
import android.widget.Button;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
Button btnStart, btnStop, btnBound;
MyBoundService myBoundService;
boolean isBound = false;
@Override

protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
btnStart = findViewById(R.id.btnStart);
btnStop = findViewById(R.id.btnStop);
btnBound = findViewById(R.id.btnBound);
// Start Service
btnStart.setOnClickListener(v -> {
```

Intent intent = new Intent(

this,

```
MyStartedService.class
);
startService(intent);
});
// Stop Service
btnStop.setOnClickListener(v -> {
Intent intent = new Intent(
this,
MyStartedService.class
);
stopService(intent);
});
// Bind Service
btnBound.setOnClickListener(v -> {
Intent intent = new Intent(
this,
MyBoundService.class
);
bindService(
intent,
serviceConnection,
BIND_AUTO_CREATE
);
});
}
// Service Connection
ServiceConnection serviceConnection = new ServiceConnection() {
@Override
public void onServiceConnected(
ComponentName name,
IBinder service) {
MyBoundService.LocalBinder binder =
(MyBoundService.LocalBinder) service;
myBoundService = binder.getService();
isBound = true;
Toast.makeText(
MainActivity.this,
myBoundService.getMessage(),
Toast.LENGTH_SHORT
).show();
}
@Override
public void onServiceDisconnected(ComponentName name) {
isBound = false;
}
};
@Override
protected void onDestroy() {
super.onDestroy();
if (isBound) {
unbindService(serviceConnection);
isBound = false;
}
}
```

```
}
```

Add Services in AndroidManifest.xml (in Android Project Panel).

Open:

app→ Manifests → AndroidManifest.xml

Inside: <application>

Add:->

<service android:name=".MyStartedService"/>

<service android:name=".MyBoundService"/>

8. Develop an Android application implementing Notifications using Notification Manager.

Project Name: NotificationDemo

File Name: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:gravity="center">
<Button
android:id="@+id/btnNotify"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Show Notification"/>
</LinearLayout>
```

Add Permission in Manifest,

Open: AndroidManifest.xml

Add above <application>:

<uses-permission android:name="android.permission.POST_NOTIFICATIONS”/>

File Name: MainActivity.java

```
package com.example.notificationdemo;
import android.Manifest;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.content.pm.PackageManager;
import android.os.Build;
import android.os.Bundle;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.NotificationCompat;
public class MainActivity extends AppCompatActivity {
private static final String CHANNEL_ID = "MyChannel";
private static final int NOTIFICATION_PERMISSION_CODE = 100;
private Button btnNotify;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);


// Find the button from activity_main.xml
btnNotify = findViewById(R.id.btnNotify);
// Create notification channel
createNotificationChannel();
// Request notification permission for Android 13+
requestNotificationPermission();
// Button click
btnNotify.setOnClickListener(v -> {
showNotification();
});
}
// Request notification permission
private void requestNotificationPermission() {
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
if (checkSelfPermission(Manifest.permission.POST_NOTIFICATIONS)
!= PackageManager.PERMISSION_GRANTED) {
```

requestPermissions(

```
new String[]{Manifest.permission.POST_NOTIFICATIONS},
NOTIFICATION_PERMISSION_CODE

);
 }
 }
 }
 // Create notification channel
 private void createNotificationChannel() {
 if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
NotificationChannel channel = new NotificationChannel(
CHANNEL_ID,
"My Channel",
NotificationManager.IMPORTANCE_DEFAULT
);
channel.setDescription("Notification Channel for Demo");
NotificationManager manager =
getSystemService(NotificationManager.class);
if (manager != null) {
manager.createNotificationChannel(channel);
 }
 }
 }

 // Show notification
 private void showNotification() {
 // Check permission for Android 13+
 if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
 if (checkSelfPermission(Manifest.permission.POST_NOTIFICATIONS)
 != PackageManager.PERMISSION_GRANTED) {
 requestNotificationPermission();
 return;
 }
 }
NotificationCompat.Builder builder =
new NotificationCompat.Builder(this, CHANNEL_ID)
.setSmallIcon(android.R.drawable.ic_dialog_info)
.setContentTitle("Notification Example")
.setContentText("Hello from Android Notification")
.setPriority(NotificationCompat.PRIORITY_DEFAULT)
.setAutoCancel(true);
NotificationManager manager =
(NotificationManager) getSystemService(NOTIFICATION_SERVICE);

if (manager != null) {
manager.notify(1, builder.build());
 }
 }
 }
```

9. Develop an Android application implementing Static and Dynamic Broadcast Receivers.

Project Name: BroadcastDemo

File Name: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:gravity="center">
<TextView
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Broadcast Receiver Example"
android:textSize="22sp"/>
</LinearLayout>
```

Create a Broadcast Receiver Class.

Right Click:-> java package → New → Java Class Class Name: MyReceiver

```
package com.example.broadcastdemo;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.widget.Toast;
public class MyReceiver extends BroadcastReceiver {
@Override
public void onReceive(Context context, Intent intent) {
Toast.makeText(
```

context,

"Broadcast Received",

Toast.LENGTH_SHORT

).show();

```
}
}
```

Static Broadcast Receiver.

Open:AndroidManifest.xml

Inside: <application>

Add:

<receiver

```
android:name=".MyReceiver"
android:exported="true">
<intent-filter>
<action android:name="android.intent.action.AIRPLANE_MODE" />
</intent-filter>
</receiver>
```

Dynamic Broadcast Receiver.

File Name: MainActivity.java

```
package com.example.broadcastdemo;
import android.content.IntentFilter;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;
public class MainActivity extends AppCompatActivity {
MyReceiver myReceiver = new MyReceiver();
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
// Dynamic Receiver
IntentFilter filter = new IntentFilter();
filter.addAction("android.intent.action.AIRPLANE_MODE");
```

ContextCompat.registerReceiver(

this,

myReceiver,

filter,

ContextCompat.RECEIVER_EXPORTED

);

```
}
@Override
protected void onDestroy() {
super.onDestroy();
unregisterReceiver(myReceiver);
}
}
```

Turn on airplane mode

Database and Networking

10. Develop an Android application implementing SQLite database CRUD operations.

ProjectName: SQLLiteCRUDDemo

File Name: activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="20dp">
<EditText
android:id="@+id/etName"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Enter Name"/>
<EditText
android:id="@+id/etNewName"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Enter New Name"/>
<Button
android:id="@+id/btnInsert"
android:layout_width="match_parent"
android:layout_height="wrap_content"


android:text="Insert"/>
<Button
android:id="@+id/btnView"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="View Data"/>
<Button
android:id="@+id/btnUpdate"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Update"/>
<Button
android:id="@+id/btnDelete"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Delete"/>
<TextView
android:id="@+id/txtResult" android:layout_width="match_parent"
android:layout_height="wrap_content"
android:textSize="18sp"
android:paddingTop="20dp"/>
</LinearLayout>
```

Create a Database Helper Class.

Right Click:

java package → New → Java Class

Class Name: DBHelper

```
package com.example.sqllitecruddemo;
import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
```

```
import android.database.sqlite.SQLiteOpenHelper;
public class DBHelper extends SQLiteOpenHelper {
public DBHelper(Context context) {
super(context, "StudentDB", null, 1);
}
@Override
public void onCreate(SQLiteDatabase db) {
db.execSQL("CREATE TABLE student(name TEXT)");
}
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
db.execSQL("DROP TABLE IF EXISTS student");
```

onCreate(db);

```
}
// Insert
public void insertData(String name) {
SQLiteDatabase db = this.getWritableDatabase();
ContentValues values = new ContentValues();
values.put("name", name);
db.insert("student", null, values);
}
// View
public String viewData() {
SQLiteDatabase db = this.getReadableDatabase();
Cursor cursor = db.rawQuery("SELECT * FROM student", null);
StringBuilder data = new StringBuilder();
while (cursor.moveToNext()) {
data.append(cursor.getString(0)).append("\n");
}
cursor.close();
```

return data.toString();

```
}
// Update
public void updateData(String oldName, String newName) {
SQLiteDatabase db = this.getWritableDatabase();
ContentValues values = new ContentValues();
values.put("name", newName);
```

db.update(

"student",

values,

"name=?",

new String[]{oldName}

);

```
}
// Delete
public void deleteData(String name) {
SQLiteDatabase db = this.getWritableDatabase();
```

db.delete(

"student",

"name=?",

new String[]{name}

);

```
}
}
```

MainActivity.java

```
package com.example.sqllitecruddemo;
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
EditText etName, etNewName;
TextView txtResult;
Button btnInsert, btnView, btnUpdate, btnDelete;
```

```
DBHelper dbHelper;

@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
etName = findViewById(R.id.etName);
etNewName = findViewById(R.id.etNewName);
txtResult = findViewById(R.id.txtResult);
btnInsert = findViewById(R.id.btnInsert);
btnView = findViewById(R.id.btnView);
btnUpdate = findViewById(R.id.btnUpdate);
btnDelete = findViewById(R.id.btnDelete);
dbHelper = new DBHelper(this);
// Insert
btnInsert.setOnClickListener(v -> {
String name = etName.getText().toString();
dbHelper.insertData(name);
Toast.makeText(
```

this,

"Data Inserted",

Toast.LENGTH_SHORT

).show();

```
});
// View
btnView.setOnClickListener(v -> {
String data = dbHelper.viewData();
txtResult.setText(data);
});
// Update
btnUpdate.setOnClickListener(v -> {
String oldName = etName.getText().toString();
String newName = etNewName.getText().toString();
dbHelper.updateData(oldName, newName);
Toast.makeText(
```

this,

"Data Updated",

Toast.LENGTH_SHORT

).show();

```
});

// Delete
btnDelete.setOnClickListener(v -> {
String name = etName.getText().toString();
dbHelper.deleteData(name);
Toast.makeText(
```

this,

"Data Deleted",

Toast.LENGTH_SHORT

).show();

```
});
}
}
```

11. Develop an Android application integrating SQLite with ListView or RecyclerView.

Project Name : SQLiteCRUDDemo

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="20dp">
<EditText
```

```
android:id="@+id/etName"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Enter Name" />
<Button
android:id="@+id/btnInsert"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="Insert" />


<Button
android:id="@+id/btnView"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="View Data" />
<ListView
android:id="@+id/listView"
android:layout_width="match_parent"
android:layout_height="wrap_content" />
</LinearLayout>
```

Create New Java Class in Package Folder

Name as: DBHelper

```
package com.example.sqlitecruddemo;
import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;
import java.util.ArrayList;
public class DBHelper extends SQLiteOpenHelper {
public DBHelper(Context context) {
```

super(context, "StudentDB", null, 1);

```
}
@Override
public void onCreate(SQLiteDatabase db) {
db.execSQL("CREATE TABLE student(name TEXT)");
}
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
// Database upgrade code goes here
}
// Insert Data
public void insertData(String name) {
SQLiteDatabase db = this.getWritableDatabase();
ContentValues values = new ContentValues();
values.put("name", name);
db.insert("student", null, values);
}
// Get All Data
public ArrayList<String> getData() {
```

ArrayList<String> list = new ArrayList<>();

SQLiteDatabase db = this.getReadableDatabase();

Cursor cursor = db.rawQuery("SELECT name FROM student", null);

while (cursor.moveToNext()) {

list.add(cursor.getString(0));

```
}
cursor.close();
return list;
}
}
```

MainActivity.java

```
package com.example.sqlitecruddemo;
import android.os.Bundle;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ListView;
import android.widget.Toast;


import androidx.appcompat.app.AppCompatActivity;
import java.util.ArrayList;
```

```
public class MainActivity extends AppCompatActivity {
EditText etName;
Button btnInsert, btnView;
ListView listView;
DBHelper dbHelper;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
etName = findViewById(R.id.etName);
btnInsert = findViewById(R.id.btnInsert);
btnView = findViewById(R.id.btnView);
listView = findViewById(R.id.listView);
dbHelper = new DBHelper(this);


// Insert Data
btnInsert.setOnClickListener(v -> {
String name = etName.getText().toString();
dbHelper.insertData(name);
Toast.makeText(
```

this,

"Data Inserted",

Toast.LENGTH_SHORT

).show();

etName.setText("");

```
});
// View Data
btnView.setOnClickListener(v -> {
ArrayList<String> list = dbHelper.getData();
```

ArrayAdapter<String> adapter = new ArrayAdapter<>(

this,

android.R.layout.simple_list_item_1,

list

);

listView.setAdapter(adapter);

```
});
}
}
```

12. Develop an Android application to fetch and parse JSON data from a web

PROJECTNAME: JSONDEMO

Add Internet Permission.

AndroidManifest.xml

Add following above: <application> tag

<uses-permission android:name="android.permission.INTERNET"/>

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="20dp">
<Button
android:id="@+id/btnFetch"
android:layout_width="match_parent"
android:layout_height="wrap_content"


android:text="Fetch JSON Data" />
<TextView
android:id="@+id/txtResult"
android:layout_width="match_parent"
```

```
android:layout_height="wrap_content"
android:layout_marginTop="20dp"
android:textSize="18sp"
android:textColor="#000000" />
</LinearLayout>
```

MainActivity.java

```
package com.example.jsondemo;
package com.example.jsondemo;
import android.os.Bundle;
import android.widget.Button;
import android.widget.TextView;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;
import org.json.JSONObject;
import androidx.appcompat.app.AppCompatActivity;


public class MainActivity extends AppCompatActivity {
Button btnFetch;
TextView txtResult;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
btnFetch = findViewById(R.id.btnFetch);
txtResult = findViewById(R.id.txtResult);
btnFetch.setOnClickListener(v -> {
fetchData();
});
}


private void fetchData() {
new Thread(() -> {
try {
// Web Service URL
```

URL url = new

URL("https://jsonplaceholder.typicode.com/posts/1");

HttpURLConnection connection =

(HttpURLConnection) url.openConnection();

connection.setRequestMethod("GET");

connection.setRequestMethod("GET");

BufferedReader reader = new BufferedReader(new

InputStreamReader(

connection.getInputStream()

)

);

StringBuilder result = new StringBuilder();

String line;

while ((line = reader.readLine()) != null) {

result.append(line);

```
}
reader.close();
// JSON Parsing
```

JSONObject jsonObject =

new JSONObject(result.toString());

String title =

jsonObject.getString("title");

String body =

jsonObject.getString("body");

```
// Display Data
runOnUiThread(() -> {
```

txtResult.setText(

"Title: " + title +

"\n\nBody: " + body

);

```
});
connection.disconnect();
} catch (Exception e) {
e.printStackTrace();
runOnUiThread(() -> {
```

txtResult.setText(

"Error: " + e.getMessage()

);

```
});
}
```

}).start();

```
}
}
```

https://jsonplaceholder.typicode.com/posts/1

Concurrency, Media and Security

13. Develop an Android application demonstrating Threads and Handlers for background

processing.

Project Name: ThreadHandlerDemo

Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:gravity="center"
android:padding="20dp">
<Button
android:id="@+id/btnStart"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Start Background Task"/>
<TextView
android:id="@+id/txtResult"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:textSize="22sp"
android:paddingTop="20dp"/></LinearLayout>
```

MainActivity.java

```
package com.example.threadhandlerdemo;
import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;
import android.widget.Button;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
Button btnStart;
TextView txtResult;
Handler handler = new Handler(Looper.getMainLooper());
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
```

```
btnStart = findViewById(R.id.btnStart);
txtResult = findViewById(R.id.txtResult);
```

btnStart.setOnClickListener(v -> {

```
// Background Thread
Thread thread = new Thread(() -> {
try {
// Simulate Background Work
Thread.sleep(3000);
} catch (Exception e) {
e.printStackTrace();
}
// Update UI using Handler
handler.post(() -> {
txtResult.setText("Background Task Completed");
});
});
thread.start();
});
}
}
```

14. Develop an Android application implementing Media APIs and Telephone APIs.

Project Name: MediaTelephoneDemo

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:gravity="center"
android:padding="20dp">
<Button
android:id="@+id/btnPlay"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Play Music"/>
<Button
android:id="@+id/btnDial"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="Open Dialer"
android:layout_marginTop="20dp"/> </LinearLayout>
```

Add Audio File.

Create New folder in Android Project Panel: res raw

Download or use pre-existing small size MP3 file inside raw folder: music.mp3

MainActivity.java.

```
package com.example.mediatelephonedemo;
import android.content.Intent;
import android.media.MediaPlayer;
import android.net.Uri;
import android.os.Bundle;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
Button btnPlay, btnDial;
MediaPlayer mediaPlayer;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
btnPlay = findViewById(R.id.btnPlay);
btnDial = findViewById(R.id.btnDial);
// Media API
mediaPlayer = MediaPlayer.create(this, R.raw.music);


// Play music
btnPlay.setOnClickListener(v -> {
mediaPlayer.start();
});
// Open phone dialer
```

```
btnDial.setOnClickListener(v -> {
Intent intent = new Intent(Intent.ACTION_DIAL);
intent.setData(Uri.parse("tel:9876543210"));
startActivity(intent);
});
}
@Override
protected void onDestroy() {
super.onDestroy();
if (mediaPlayer != null) {
mediaPlayer.release();
mediaPlayer = null;
}
}
}
```

15. Develop an Android application implementing Runtime Permissions and security

handling in AndroidManifest.xml

Project Name:PermissionDemo

Add Permission in AndroidManifest.xml.

Open:

AndroidManifest.xml

Add below code lines above: <application> tag

<uses-permission android:name="android.permission.CAMERA"/>

<uses-feature android:name="android.hardware.camera"

```
android:required="false" />
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:gravity="center">
<Button
android:id="@+id/btnPermission"
android:layout_width="wrap_content"
android:layout_height="wrap_content"


android:text="Request Camera Permission"/>
</LinearLayout>
```

MainActivity.java

```
package com.example.permissiondemo;
import android.content.pm.PackageManager;
import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
public class MainActivity extends AppCompatActivity {
Button btnPermission;
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_main);
btnPermission = findViewById(R.id.btnPermission);
```

btnPermission.setOnClickListener(v -> {

checkPermission();

```
});
}
private void checkPermission() {
// Check if Camera permission is already granted
if (ContextCompat.checkSelfPermission(
```

this,

android.Manifest.permission.CAMERA

```
) == PackageManager.PERMISSION_GRANTED) {
Toast.makeText(
this,
"Permission Already Granted",
Toast.LENGTH_SHORT
).show();
} else {
 // Request Camera permission
ActivityCompat.requestPermissions(
this,
new String[]{
android.Manifest.permission.CAMERA

},
);
 }
 }
 // Permission Result
 @Override
 public void onRequestPermissionsResult(
int requestCode,
String[] permissions,
int[] grantResults
) {
super.onRequestPermissionsResult(
requestCode,
permissions,
grantResults
);
if (requestCode == 1) {
if (grantResults.length > 0
&& grantResults[0] == PackageManager.PERMISSION_GRANTED) {
Toast.makeText(
this,

"Permission Granted",
Toast.LENGTH_SHORT
).show();
} else {
Toast.makeText(
this,
"Permission Denied",
Toast.LENGTH_SHORT
).show();
 }
 }
 }
 }
```